

Task 1a: The function creates an empty matrix of length $(V+1)$ and ^xwidth $(V+1)$ size and then uses a for loop to change the weight of the $[U][V]$ index of the matrix to w U being the predecessor and V being the adjacent vertex. ~~The~~ U, V, w is taken from a list with nested tuples containing (U, V, w) each tuple represents an edge and its info. $\rightarrow$ Adjacency Matrix

Task 1b: Creates an Adjacency list which contains nested lists with tuples within each index of the list represents an index of a vertex which at which there is a list which contains the adjacent vertex and the weight of the edge

$$\left[\underbrace{[(v_1, w_1), (v_2, w_2)]}_{U_1}, \underbrace{[(v_1, w_1)]}_{U_2}, \underbrace{[(v_1, w_1)]}_{U_3} \right]$$

Task 2: The graph is traversed Breadth wise that is we will get a source and we will traverse all the adjacent vertices of the source and then we select one of the vertices of the source and traverse all its vertices. This is done by implementing a queue ~~and it~~ At first we append the source in the queue and then dequeue it and append all of its vertices ~~and~~ and repeat until the queue is empty. We also have a state list and we set the state to [1] if a vertex is Task 3 visited and we set the state to [2] if ^{adjacent} the vertices of that vertex is visited as well.

Task 3: The graph is traversed depth wise. The function recursively visits the graph to the depth by calling the function on each vertex. The state of vertex is set to 1 when its visited. when there is no where to visit the function backtracks to the previous vertex and visits other adjacent vertices of that vertex. when all the vertices of a vertex is visited its state is set to 2.

Task 4: Runs a DFS on the graph with a

Slight tweak. if ~~the state~~ a visited graph is visited again i.e. its state is 2 then it's a cycle else if the DFS runs without ~~any~~ meeting the condition above it's not a cycle.

Task 5 Runs a tweaked version of BFS with distance and parent lists which sets the distance of adjacent vertices $\text{distance}[v] = \text{distance}[u] + 1$ and $\text{parent}[v] = u$. The ~~graph then uses~~ parent of source is set to -1. The code then uses recursion to find the shortest path to the destination and uses $\text{distance}[\text{destination}]$ to get the time of visiting a said node.

Task 6 : For each index of the map the graph runs DFS and checks for the max amount of gold retrieved. In the DFS code the recursion occurs on 4 adjacent sides top, right, up and bottom the code also implements a flag matrix to keep track the visited indexes the code then back tracks ~~to~~ and returns the total diamonds found by adding diamonds found on each traversal and returns the maximum number of diamonds possible to retrieve